

AI Agents

Architecture, tool use, multi-agent systems & interview essentials

1. What is an AI Agent?

- An AI agent is a system that uses an LLM as a **reasoning engine** to autonomously decide a sequence of actions — calling tools, querying data, or invoking other models — to achieve a goal, rather than just producing a single one-shot text response.
- Key distinction from a plain chatbot: an agent **observes results and decides what to do next** in a loop, instead of just answering directly.
- Core loop: **Perceive** → **Reason/Plan** → **Act (use a tool)** → **Observe result** → **repeat until goal is met**.

2. Core Components

- **LLM (the "brain")**: Performs reasoning, planning, and decides which action/tool to invoke next.
- **Tools / Actions**: Functions the agent can call — web search, code execution, database queries, APIs, file I/O — extending the LLM beyond text generation.
- **Memory**: Short-term (conversation/context window) and long-term (vector store or database) memory so the agent can recall past interactions or facts across sessions.
- **Planning module**: Breaks a complex goal into smaller steps/sub-tasks, sometimes with explicit re-planning if a step fails.
- **Orchestrator/controller**: The loop logic that decides when to stop, retries failed actions, and enforces limits (max steps, timeouts, guardrails).

3. Common Agent Architectures

Pattern	How it Works
ReAct (Reason + Act)	Interleaves reasoning traces ("Thought") with actions ("Act") and observations in a loop — the classic agent pattern.
Plan-and-Execute	Generates a full multi-step plan up front, then executes each step, optionally re-planning if a step fails.
Reflexion / Self-Refine	After acting, the agent critiques its own output/result and revises its approach — adds a self-correction loop.
Tree of Thoughts	Explores multiple reasoning branches in parallel and evaluates/backtracks, rather than one linear chain.
Orchestrator–Worker	A top-level agent decomposes the task and delegates sub-tasks to specialized worker agents/tools, then aggregates results.

4. Tool Use (Function Calling)

- The developer defines a set of tools/functions with a name, description, and structured input schema (e.g. JSON schema).
- The LLM is given these definitions and, based on the user request, outputs a structured call (function name + arguments) instead of free text when a tool is needed.
- The application code executes the actual function/API call and returns the result to the LLM as an "observation," which the model then reasons over to decide its next step or final answer.
- This is the mechanism that turns a text-only LLM into something that can browse, run code, query a database, or trigger real-world actions.

5. Memory in Agents

- **Short-term memory**: The current conversation/context window — the agent's immediate working memory, which is finite.
- **Long-term memory**: Persisted externally (often a vector store) so the agent can retrieve relevant past facts/interactions — effectively RAG applied to the agent's own history.
- **Episodic vs. semantic memory**: Episodic = specific past events/interactions; semantic = general learned facts/preferences distilled from those events.

6. Multi-Agent Systems

- **Why multi-agent**: Splitting a complex task across specialized agents (e.g. "researcher," "coder," "reviewer") can improve quality and modularity versus one monolithic agent trying to do everything.
- **Orchestration patterns**: Sequential hand-off, hierarchical (manager delegates to workers), or debate/critique (agents challenge each other's outputs to catch errors).
- **Trade-off**: Better task decomposition and specialization, at the cost of higher latency, cost, and coordination complexity (e.g. agents disagreeing or looping).

7. Popular Agent Frameworks

Framework	Focus
LangChain / LangGraph	General-purpose LLM app framework; LangGraph adds explicit graph-based control flow for agents.
LlamaIndex Agents	Agent workflows built around data indexing/retrieval-centric applications.
CrewAI	Role-based multi-agent orchestration (define agents with roles, goals, and tasks).
AutoGen	Microsoft's framework for multi-agent conversation/collaboration patterns.
OpenAI / Anthropic Agent SDKs	Native function-calling & agent-loop primitives from the model providers themselves.

8. Challenges & Failure Modes

- **Reliability:** Errors compound across multi-step loops — a small mistake early on can derail the whole task.
- **Infinite/repetitive loops:** Agent gets stuck retrying the same failed action; needs step limits and loop detection.
- **Hallucinated tool calls:** Model invents a tool/parameter that doesn't exist or passes malformed arguments.
- **Cost & latency:** Each reasoning + tool-call round trip adds tokens and time; long agent loops can get expensive.
- **Evaluation difficulty:** Hard to benchmark open-ended, multi-step tasks compared to single-turn QA.
- **Safety/guardrails:** Agents that can take real-world actions (send emails, spend money, modify files) need permission checks, human-in-the-loop approval for risky actions, and sandboxing.

9. Quick-Fire Interview Answers

- **Agent vs. plain LLM call?** A plain call is one-shot text-in/text-out; an agent runs a loop where the LLM decides actions, observes results, and iterates until the goal is done or a limit is hit.
- **Agent vs. RAG?** RAG is a (usually single-step) retrieval-then-generate pattern; an agent can use RAG as one of many tools within a broader multi-step reasoning loop.
- **How do you keep an agent from looping forever?** Max iteration/step limits, timeouts, explicit stop conditions, and monitoring for repeated identical actions.
- **How do you make tool calls reliable?** Strict schemas/validation on tool inputs, clear tool descriptions, retries with error feedback to the model, and constraining which tools are available per step.
- **One-line summary:** An AI agent wraps an LLM in a perceive-plan-act-observe loop with tools and memory, turning it from a text generator into a system that can autonomously pursue multi-step goals.